# Применение depthwise separable convolutions для ускорения инференса CNN на CPU

Курсовая работа

Станиславчук Сергей

3 июня 2026

# Введение

**Проблема**
Глубокие CNN (ResNet-18, ResNet-50, . . . ) достигают высокой точности, но содержат **десятки миллионов параметров** и требуют больших вычислительных затрат. Это ограничивает их применение на мобильных и встраиваемых устройствах.

# Введение

**Проблема**
Глубокие CNN (ResNet-18, ResNet-50, . . . ) достигают высокой точности, но содержат **десятки миллионов параметров** и требуют больших вычислительных затрат. Это ограничивает их применение на мобильных и встраиваемых устройствах.

**Решение**
Замена *стандартных* свёрток на *depthwise-разделимые* (в стиле MobileNet) позволяет резко сократить число параметров и ускорить инференс при минимальной потере точности.

# Введение

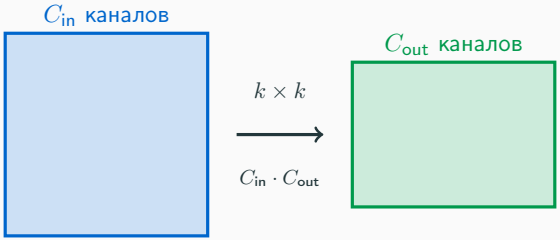
**Проблема**
Глубокие CNN (ResNet-18, ResNet-50, . . . ) достигают высокой точности, но содержат **десятки миллионов параметров** и требуют больших вычислительных затрат. Это ограничивает их применение на мобильных и встраиваемых устройствах.

**Решение**
Замена *стандартных* свёрток на *depthwise-разделимые* (в стиле MobileNet) позволяет резко сократить число параметров и ускорить инференс при минимальной потере точности.

**Цель работы**
Реализовать и сравнить ResNet-18 со стандартными (1) и depthwise-разделимыми (2) свёртками на датасете CIFAR-10 по трём метрикам:

- число параметров и размер модели
- задержка инференса (latency) на CPU

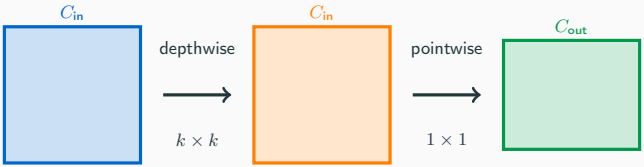
# Стандартная свёртка




**Параметры:** $k^2 \cdot C_{\text{in}} \cdot C_{\text{out}}$

**Вычисления:** $k^2 \cdot C_{\text{in}} \cdot C_{\text{out}} \cdot H \cdot W$

# Depthwise-разделимая свёртка

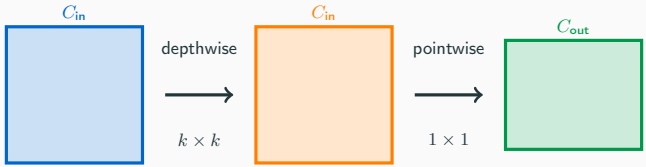



**Depthwise:** $k^2 \cdot C_{\text{in}}$ (каждый канал — отдельно)

**Pointwise:** $C_{\text{in}} \cdot C_{\text{out}}$ (смешивание каналов)

**Всего:** $k^2 \cdot C_{\text{in}} + C_{\text{in}} \cdot C_{\text{out}}$

# Depthwise-разделимая свёртка




**Depthwise:** $k^2 \cdot C_{\text{in}}$ (каждый канал — отдельно)

**Pointwise:** $C_{\text{in}} \cdot C_{\text{out}}$ (смешивание каналов)

**Всего:** $k^2 \cdot C_{\text{in}} + C_{\text{in}} \cdot C_{\text{out}}$

**Сравнение по параметрам ($k = 3$)**

$$\frac{\text{DW-Sep}}{\text{Standard}} = \frac{9C_{\text{in}} + C_{\text{in}}C_{\text{out}}}{9C_{\text{in}}C_{\text{out}}} \approx \frac{1}{C_{\text{out}}} + \frac{1}{9}$$

При типичном $C_{\text{out}} \geq 64$ — сокращение в **5–10 раз**.

# Архитектура ResNet-18 (CIFAR-10)

| Слой | Выход | Блоки |
|---|---|---|
| conv1 | $32 \times 32$ | $3 \times 3,\ 64$ |
| layer1 | $32 \times 32$ | $\begin{bmatrix} 3 \times 3,\ 64 \\ 3 \times 3,\ 64 \end{bmatrix} \times 2$ |
| layer2 | $16 \times 16$ | $\begin{bmatrix} 3 \times 3,\ 128 \\ 3 \times 3,\ 128 \end{bmatrix} \times 2$ |
| layer3 | $8 \times 8$ | $\begin{bmatrix} 3 \times 3,\ 256 \\ 3 \times 3,\ 256 \end{bmatrix} \times 2$ |
| layer4 | $4 \times 4$ | $\begin{bmatrix} 3 \times 3,\ 512 \\ 3 \times 3,\ 512 \end{bmatrix} \times 2$ |
| avg-pool + fc | $1 \times 1$ | 10 классов |

# Датасет: CIFAR-10

**Характеристики**

- **10 классов**: самолёт, авто, птица, кот, олень, собака, лягушка, лошадь, корабль, грузовик.
- **60 000** цветных изображений $32 \times 32$.
- 50 000 — обучение, 10 000 — тест.

**Аугментации**

- RandomCrop(32, padding=4)
- RandomHorizontalFlip
- Нормализация:
  $\mu = (0.4914, 0.4822, 0.4465)$
  $\sigma = (0.2470, 0.2435, 0.2616)$

# Методология обучения

**Гиперпараметры**

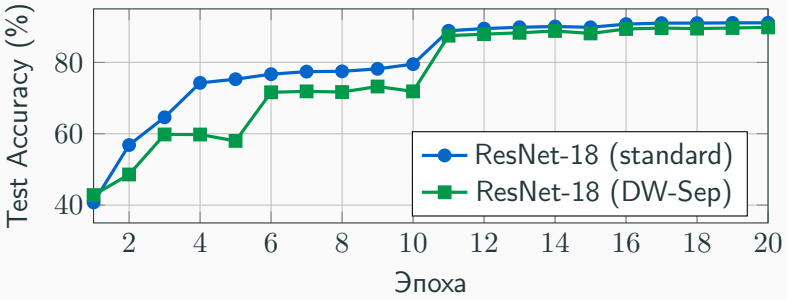
- Оптимизатор: SGD, момент 0.9
- Weight decay: $5 \cdot 10^{-4}$
- Начальный lr: 0.1
- MultiStepLR с $\gamma = 0.1$
- Этапы: на 50% и 75% эпох
- Batch size: 128

**Эксперимент**

- Устройство: CPU (Intel Core)
- Эпохи: 20
- Потоки CPU: 8
- Seed: 42 (воспроизводимость)

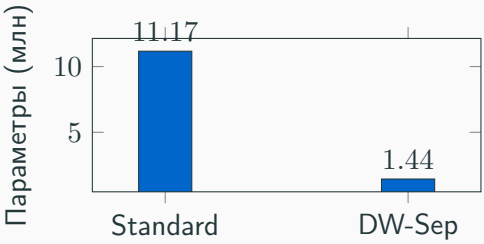
Обе модели обучались в идентичных условиях — те же данные, тот же seed, те же гиперпараметры.

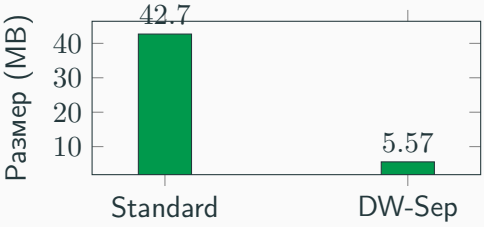
# Динамика обучения



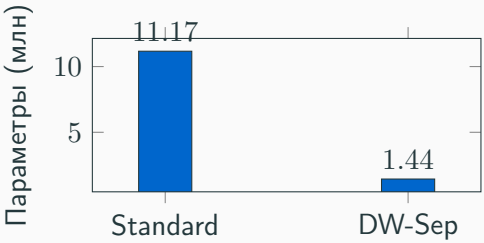

- После снижения lr на 10-й эпохе — резкий скачок точности у обеих моделей.
- Separable-версия обучается чуть медленнее, но выходит на плато рядом со standard.

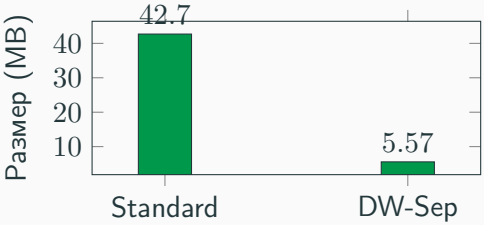
# Сравнение: параметры и размер модели



# Сравнение: параметры и размер модели

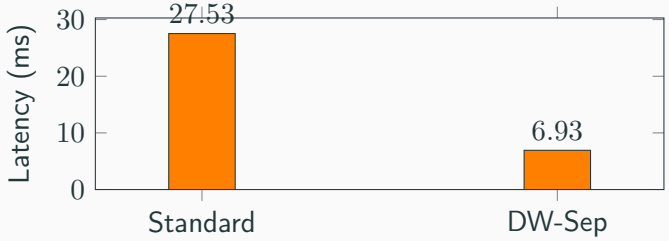



**Ключевой результат**

**Сокращение в $7.67\times$** по размеру модели и **в $7.76\times$** по числу параметров.

# Сравнение: задержка инференса (CPU)

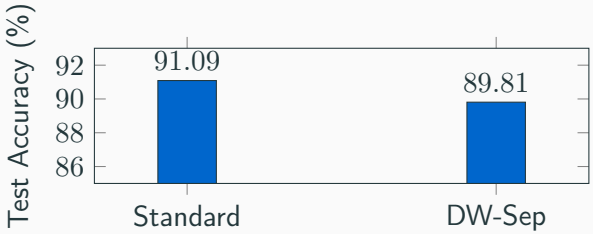



**Детали бенчмарка**

- Устройство: CPU (8 потоков), batch $= 1$, размер входа $3 \times 32 \times 32$.
- Прогрев: 50 итераций, замеры: 300 итераций.
- Стандартная модель: $27.53 \pm 2.33$ мс.
- Separable-модель: $6.93 \pm 0.16$ мс.

**Ускорение в $3.97\times$** на CPU-инференсе.

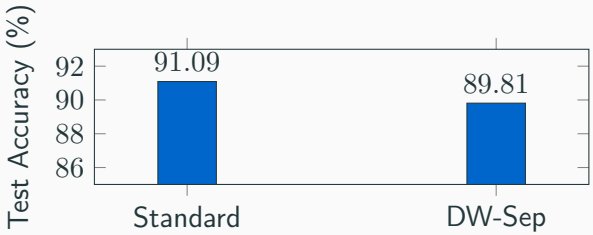
# Сравнение: точность классификации




**Результат**

- Standard: **91.09%**
- DW-Separable: **89.81%**
- Потеря точности: всего $\mathbf{1.28}$ процентных пункта.

# Сравнение: точность классификации




**Результат**

- Standard: **91.09%**
- DW-Separable: **89.81%**
- Потеря точности: всего $\mathbf{1.28}$ процентных пункта.

**Вывод**

Потеря $\approx 1.3$ п.п. точности при сокращении модели в **7.7 раз** и ускорении в **4 раза** — отличный компромисс.

# Сводная таблица результатов

| Модель | Параметры | Размер (MB) | Latency (ms) | Точность (%) |
|---|---|---|---|---|
| ResNet-18 (standard) | 11 173 962 | 42.70 | $27.53 \pm 2.33$ | 91.09 |
| ResNet-18 (DW-Sep) | 1 438 565 | 5.57 | $6.93 \pm 0.16$ | 89.81 |

**Производные метрики**

**Latency speedup** $\mathbf{3.97}\times$ — separable-модель быстрее

**Size ratio** $\mathbf{7.67}\times$ — separable-модель меньше

**Accuracy** $\Delta$ $+1.28$ п.п. — standard точнее

# Анализ результатов

**Почему такое большое сжатие?**

- В глубоких слоях ($C = 256, 512$) стандартная свёртка содержит $9 \cdot 256 \cdot 512 \approx 1.18$ млн параметров.
- Depthwise-разделимая: $9 \cdot 256 + 256 \cdot 512 \approx 0.13$ млн — в **9 раз** меньше.
- Эффект накапливается через 17 свёрточных слоёв ResNet-18.

# Анализ результатов

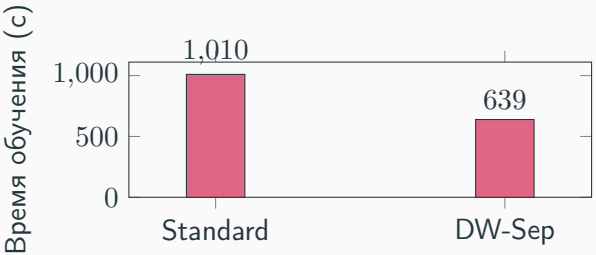
**Почему такое большое сжатие?**

- В глубоких слоях ($C = 256, 512$) стандартная свёртка содержит $9 \cdot 256 \cdot 512 \approx 1.18$ млн параметров.
- Depthwise-разделимая: $9 \cdot 256 + 256 \cdot 512 \approx 0.13$ млн — в **9 раз** меньше.
- Эффект накапливается через 17 свёрточных слоёв ResNet-18.

**Почему точность почти не падает?**

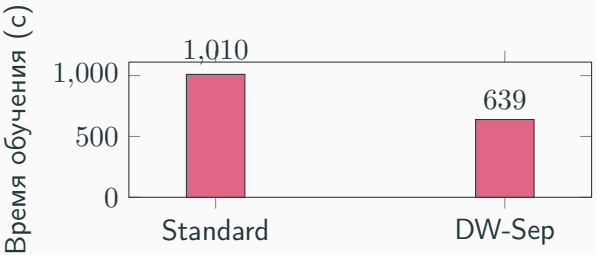
- Depthwise-свёртка сохраняет пространственную структуру (те же $k \times k$).
- Pointwise $1 \times 1$ даёт полное смешивание каналов.
- Skip-соединения (ResNet) компенсируют снижение ёмкости.

# Время обучения




- Стандартная модель: $\approx 1010$ с (20 эпох, 8 CPU-потоков).
- Separable-модель: $\approx 639$ с.
- Обучение separable-версии на **37% быстрее** — меньше параметров $\Rightarrow$ меньше градиентов $\Rightarrow$ меньше вычислений.

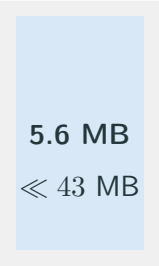
# Время обучения




- Стандартная модель: $\approx 1010$ с (20 эпох, 8 CPU-потоков).
- Separable-модель: $\approx 639$ с.
- Обучение separable-версии на **37% быстрее** — меньше параметров $\Rightarrow$ меньше градиентов $\Rightarrow$ меньше вычислений.

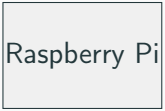
# Где это применимо?

## Мобильные устройства




- Классификация изображений
- Распознавание объектов
- On-device ML

## Встраиваемые системы




- Ограниченная память
- Нет GPU
- Низкое энергопотребление

Depthwise-разделимые свёртки — основа MobileNet, EfficientNet, Xception и других эффективных архитектур

# Заключение

1. Реализованы и обучены две версии ResNet-18 для CIFAR-10: **стандартная** и **depthwise-разделимая**.
2. Depthwise-разделимая версия:
   - в $7.7\times$ меньше параметров,
   - в $7.7\times$ меньше занимает на диске,
   - в $4.0\times$ быстрее на CPU-инференсе,
   - теряет лишь $1.28$ п.п. точности (89.81% против 91.09%).
3. Подход оправдан для сценариев с ограниченными ресурсами (мобильные и встраиваемые устройства).

**Спасибо за внимание!**